

Package Ecosystem Plan

Rene Rivera – grafikrobot@gmail.com – P1177R0, 2018-10-06 | Initial

Table of Contents

- 1. Abstract
 - 2. Introduction
 - 3. Terminology
 - 4. Current Landscape
 - 5. The Future
 - 5.1. Package Index
 - 5.2. Interfaces
 - 6. Plan
 - 6.1. Compiler API
 - 6.1.1. Standard Options
 - 6.2. Build System API
 - 6.3. PDM API
 - 6.3.1. Package Dependency
 - 6.3.2. Packages Description
 - 6.3.3. Universal Package Identifier
 - 6.4. Package Index API
 - 6.4.1. Package Query
 - 6.4.2. Package List
 - 6.5. Package Index Authority
 - 7. Acknowledgements
-

Document number	ISO/IEC/JTC1/SC22/WG21/P1177R0
Date	2018-10-06
Reply-to	Rene Rivera, grafikrobot@gmail.com
Audience	Tooling (SG15)

1. Abstract

A plan for a cohesive ecosystem of package production and consumption among all C++ tools.

2. Introduction

We've reached a point in C++ where our success has trapped us in a conundrum. The C++ Language and the standard library is a popular and high quality platform upon which to program for many domains. As such it is a popular vehicle for extensions for narrower and narrower domains. Which brings the unfortunate problem of making the management of those additions take more and more effort. The well known solution to this problem is to adopt package and dependency management as an extension to the built-in platform capabilities. Unfortunately for C++ the package and dependency management solutions to date are not interoperable nor interchangeable and hence cause confusion for users. As not only do they have to choose a compiler and environment, but now they also get to choose a package manager.

This paper intends to define a general structure of a package ecosystem wherein users can expect to consume and produce libraries with whatever C++ tools they choose in an interchangeable manner.

This paper **does not** propose to create a single standard build system, dependency manager, package manager, nor packages.

3. Terminology

Some of the terms used herein have historically varied meanings. In this paper here are what we intend when we refer to the terms.

Library

The group of code, compiled or as source, that you use directly. For example: libz, Boost MP11, and QtCore.

Package

An object that defines, and possibly implements, everything a user needs to use a library in your project. It can be a combination of some or all of: source for a library, pre-built binaries for a library, instructions for building the library, instructions for downloading binaries, enumeration of dependencies of the library.

Dependency Manager

Controls how the use of libraries in your project correlates to corresponding packages that provide those libraries to you project and how to arrange those package libraries to make them usable by your project.

Package Manager

Takes references to packages your project wants and delivers them such that the dependency manager can instruct your project to use those packages. This can include downloading from a non-local resource and building for the use case of your project.

Package and Dependency Manager

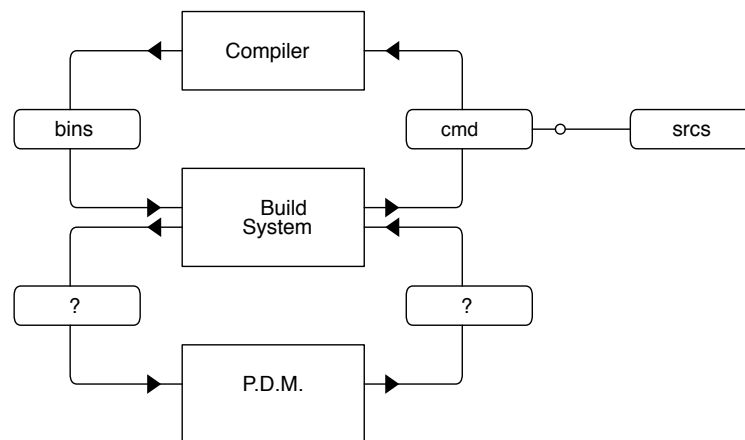
It is common, because of the close dependence between them, to combine both the dependency manager and the package manager. In this paper this is the form which we consider as it simplifies the discussion and is the form generally found in the wild.

4. Current Landscape

At this time we have three key components to our package ecosystem:

- Compiler
- Build System
- Package and Dependency Manager (PDM), and related packages

Below is a simplified view of the data flow between those components. As depending on the build system and PDM they may use additional data including source files.



Data		Process	
X Produces Y Y Accepts X	X  Y	X Uses Y	X  Y

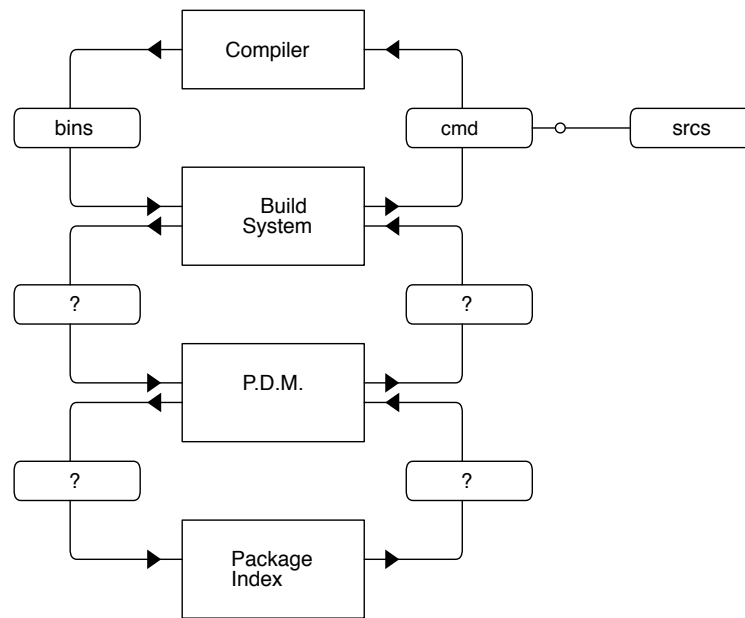
5. The Future

5.1. Package Index

One key problem with the current situation is that users consume packages from a restricted universe. When you decide to use PDM *A* you also decide to use the packages available **only** from *A*. Hence if there is a package *P* only available from PDM *B* you are back to some uncomfortable choices for your development.

That is not the only problem; There is also the issue that discovering what packages, from which PDMs, would involve going through each PDM and searching for what they have.

To resolve these we see the need for a global "Package Index". The index would contain data for each library in the C++ ecosystem. It would contain canonical information for each library such as: available packages (and the PDM of each), library name, description, license, and so on. What it would not contain would be the packages themselves. Having this would allow users to determine quickly where to get the one library they are interested in, hopefully with minimal effort.

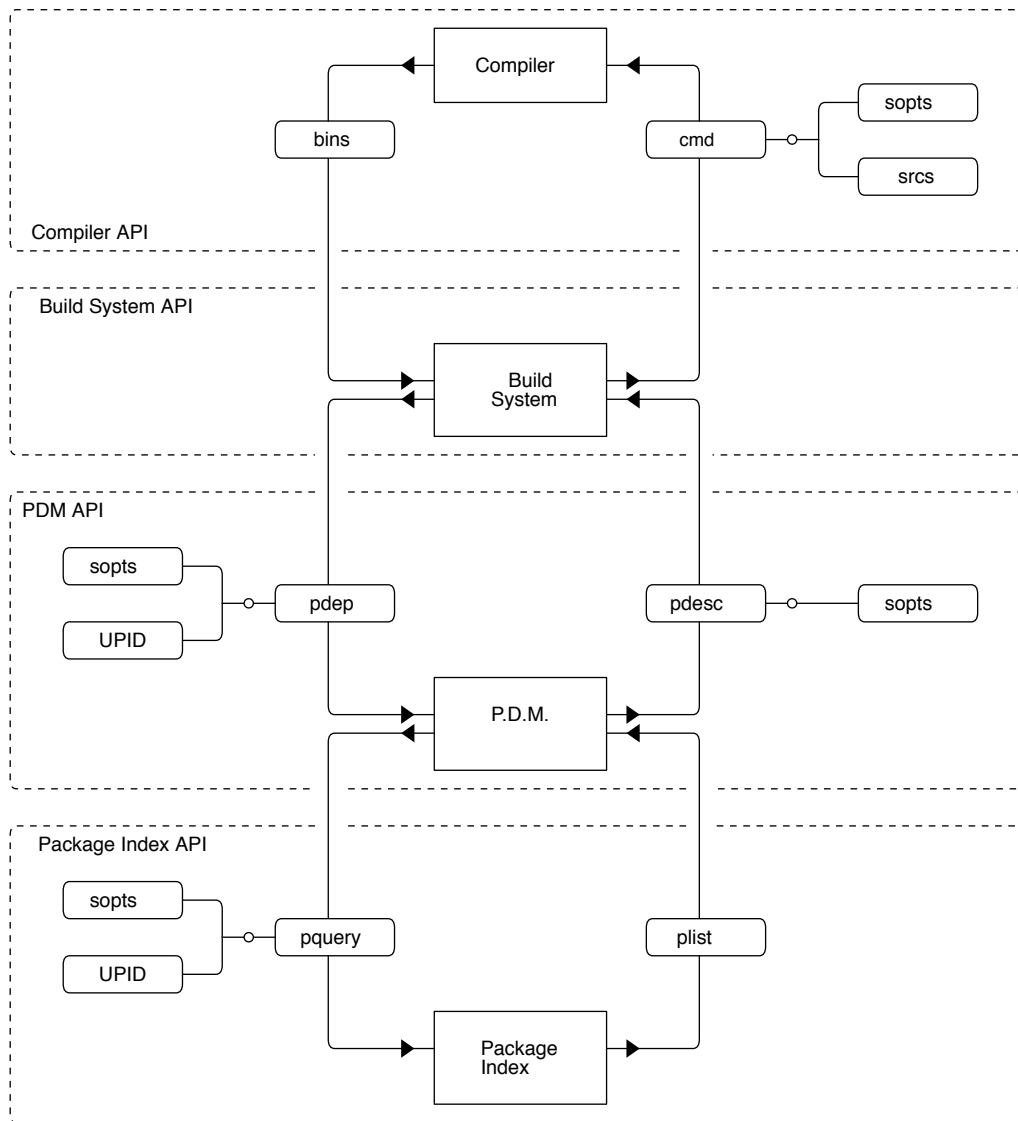


5.2. Interfaces

There is one glaring problem with the above future structure: Users are now not only dealing with how to interact with multiple compilers and build systems, but now would also have to deal with multiple PDMs. Thankfully, they might be spared having to deal with multiple package indices. We need to create standard interfaces to all our tools such that we gain the varied benefits from the common understanding they bring. What does it mean to create such standard interfaces? In terms of packages it means defining standards for:

- Invoking compilers
- Compiler responses
- Build system consumption and production of packages
- Communicating with a Package Index

We need to define all the data flow connections in our ecosystem in a standard and interchangeable manner:



In the diagram the following are used as shorthands:

<code>bins</code>	General output from compilers, might not be just binaries
<code>cmd</code>	Compiler commands
<code>srcls</code>	Language source we feed our tools with
<code>sopts</code>	Standard Options
<code>pdep</code>	Package Dependency
<code>pdesc</code>	Package Description
<code>UPID</code>	Universal Package Identifier
<code>pquery</code>	Package Query
<code>plist</code>	Package List

6. Plan

Note that this is a simplified view of the breadth of what such interfaces would need to specify. It is limited to the minimal interactions for using packages. Proposals for the individual components would be sufficiently detailed to handle the varied use cases of the tools ecosystem as a whole.

6.1. Compiler API

Goal

Define standard interfaces to control and communicate with compilers to produce data required for build systems, PDMs, source editors, and so on.

Given that compilers are at the core of our data needs we would expect this API to expand over time to address the various needs of the tools that need it.

6.1.1. Standard Options

Currently build tools, and users, need to deal with a differing set of APIs to communicate with compilers. This makes it difficult to define consistent build descriptions not just in the build system but through any tooling that needs reproducible builds.

In an ideal ecosystem all compilers would use a well known single options API that the whole ecosystem would use to interoperate between different compilers, build systems, PDMs, and other tools.

The `std::compile` proposal (P1178) covers this topic.

6.2. Build System API

Goal

Define minimal standard interface to build and define software build requirements.

6.3. PDM API

Goal

Define standard interfaces to control and communicate with PDMs to both consume and produce packages.

6.3.1. Package Dependency

Currently when using a PDM one uses a package reference particular to that PDM to indicate your dependency on a package. This makes using different PDMs difficult. It means that if you ever want to share your project with someone who uses a different PDM they need to rewrite all those package references, assuming equivalents even exist in that other PDM.

We need to define a single syntax to describe the packages we depend on. Having that would allow build systems to use PDMs interchangeably opening the entire collection of packages to users.

6.3.2. Packages Description

Like with the package dependency, we currently use different ways of describing the packages we need for any particular build of our software depending on the PDM we are using. We can't write a tool, like a build system, to communicate with PDMs one time. And like compilers we end up needing to write synthetic interfaces to each PDM on every tool.

Like having the common standard options, having common package description among PDMs allows us to write interchangeable tools on top of such an API. This API would combine the standard options and a package dependency to ask PDMs for the specific build variation we need to use. What happens behind the scenes to get you a package that matches your requirements is, as we are fond of saying, *implementation defined*. This API would: allow tool makers to "write once" to use PDMs, allow users to migrate from one PDM to another, it could even allow use of multiple PDMs simultaneously if such a need arises.

6.3.3. Universal Package Identifier

To be able to support interchangeability between different PDMs we can't be tied to their individual package references. We need a single universal package identifier (UPID) to use in our projects that can refer to the components.

6.4. Package Index API

Goal

Define standard interfaces to query and publish package records for PDMs, and users.

6.4.1. Package Query

Having a package index is no good without a consistent way of asking for packages. A standard package query that tools can use to interrogate available packages is just one of the many interface points we need in an index.

6.4.2. Package List

At minimum a package index needs to respond to queries with the set of packages available. The package list defines a standard response to that query.

6.5. Package Index Authority

Goal

Create a process for assigning a single authoritative source of package index information.

There is a key question that arises from having a package index.. Should we have one or many such indices? There are pros and cons for such a choice but there is one overwhelming concern.. Having multiple indices causes fragmentation and confusion for users. Assuming that there will be multiple parties interested in providing the index service we need to define who will be the authoritative source of this data. Hence we need to define a process that includes selection criteria, review, and official designation of who is the universal package index authority.

7. Acknowledgements

Thanks to CppCon 2017 for providing the environment for "arguments" about build systems, package managers, and dependency managers that created the impetus for this idea.

Thanks to Breno Rodrigues Guimarães, Steve Downey, and others in the CppLang Slack community who provided feedback to the draft version of this document.